



Маршрутизация.

i Система спавнера даже на бумаге кажется очень сложной. Простой спавнер, который будет брать врагов из Pool и включать их на точке А, передавать им все точки маршрута и активировать движение - сделать не сложно. Ещё создавая мобов, я знал к чему идти - на сцене будет массив Transform квадратов, которые будут передаваться во врагов через Spawner. Враги будут брать случайную точку поочередно из диапазона квадрата и идти по ним, как бы слегка отклоняясь от идеально ровного курса.

Учитывая, что уровни будут выбираться случайно, а путей может быть больше одного - нужна универсальная система, которая позволит содержать массивы таких путей, а также будет выдавать их в случайном порядке (разные волны смогут идти по разным маршрутам). Добавим сюда же возможность делить волну мобов и пускать её сразу по двум путям в случайном порядке.

Однако, может быть и такое, что если я захочу сделать карту, где два пути не будут пересекаться, а спавнер решит разделить волну пополам - пустит одну часть волны по пути А, другую по пути Б. И быть это может в самом старте игры, при наличии лишь пары башен, покрывающих только один путь. В таком случае, нужно иметь не только обширный арсенал путей и возможность пускать врагов по всем из них, но и возможность ручного управления в некоторых ситуациях. Для этого понадобится двумерный массив, где у каждой волны будет список путей.

- Если путь не указан → выбирается случайный из общего списка.
Если путь указан → спавнер пустит именно по этому маршруту.
Если путей больше 1+ → можем послать по всем сразу одну волну.

i На поздних этапах (4-5 волны) можно запускать по всем путям сразу, нужно лишь указать их в инспекторе. Если дизайн потребует, чтобы первые волны прошли именно на одном пути → такая возможность будет. Например: будет вторичный путь, на который придётся отвлечься лишь попозже, когда игрок успеет занять башни в той части карты (например на 2-3 волне). Возможно, старт в моей игре окажется слишком быстрым и тогда башен будет в любом случае хватать, а игра покажется очень лёгкой. В финале, когда всё будет работать, меня ждёт ребаланс игры.

- Простой скрипт хранения путей, который будет лежать на каждой сцене. При загрузке сцены спавнер будет находить его сам и брать данные. Хотя можно было прокидывать данные немного иным путём, но этого достаточно.

```

1 using UnityEngine;
2
3 public class SceneEnabler : MonoBehaviour
4 {
5     [Header("Paths for enemies")]
6     public Transform[][] enemyPaths;
7
8     [Header("Custom wave settings")]
9     public int[][] waveCustomPaths;
10
11     [Header("Other settings")]
12     public bool useCustomWaveSettings = false;
13
14 }

```

Сразу же сталкиваемся с проблемой, и узнаём, что Unity не умеет в сериализацию сложных данных. В нашем случае - зубчатых массивов. Он не показывает их в инспекторе. Поэтому придётся сделать обёртки:

```

1 using UnityEngine;
2
3 [System.Serializable]
4 public class EnemyPath
5 {
6     public Transform[] points;
7 }
8
9 [System.Serializable]
10 public class WavePathConfig
11 {
12     public int[] pathIndexes;
13 }
14
15 //ОБНОВЛЕННЫЙ СЦЕН-ЕНЕЙБЛЕР
16 using UnityEngine;
17
18 public class SceneEnabler : MonoBehaviour
19 {
20     [Header("Paths for enemies")]
21     public EnemyPath[] enemyPaths;
22     [Header("Custom wave settings")]
23     public WavePathConfig[] waveCustomPaths;
24     [Header("Other settings")]
25     public bool useCustomWaveSettings = false;
26
27 }

```

CustomWait.

```

1 private IEnumerator SpawnWave()
2 {
3     List<EnemyType> waveEnemies = BuildWaveEnemyList();
4

```

```

5     foreach (var enemyType in waveEnemies)
6     {
7         SpawnEnemy(enemyType, currentWavePathIndexes);
8
9         yield return new WaitForSeconds(
10             spawnDelay / SpeedGameManager.Instance.SpeedMultiplier
11         );
12     }
13 }

```

При написании SpawnerManager я столкнулся с осознанием следующей проблемы. Я использовал встроенный WaitForSeconds, куда передавал значения задержек. Однако некоторые задержки должны также учитывать текущую игровую скорость - SpeedMultiplierю. Если условный таймер пред-волны начинает отчёт от 7 секунд, то на скорости 0 он должен застыть. Изначально я не учёл этого, и на паузе UI текст таймера менялся, как и Wait.

Следующие проблемы: при делении задержки на текущую скорость игры, мы, конечно, сократим время спавна (ускорим игру). Однако если щелкать множители времени на ходу - Wait не обновится, потому что данные мы уже передали в метод и он уже выполняется.

Решение простое. Мне не понадобится вручную перевычислять какие-то данные. Я сделал свой кастомный Wait аналог:

```

1 private IEnumerator WaitScaled(float duration)
2 {
3     float timer = duration;
4
5     while (timer > 0f)
6     {
7         timer -= Time.deltaTime *
8             SpeedGameManager.Instance.SpeedMultiplier;
9         yield return null;
10    }
11 }
12 /// Меняем старую запись на нашу функцию.
13 yield return new WaitForSeconds(...); =>
14     yield return WaitScaled(...);

```

Я могу работать с таким таймером отчёта как с обычным Wait, останавливая его даже во время выполнения, потому что это IEnumerator. Теперь таймер отнимает не просто deltaTime, а ещё и умножает прошедшее время на текущую скорость игры, и если скорость будет 0 (пауза) - то и таймер перестанет отниматься, а короутина просто будет крутиться в ожидании, когда игровое время снова пойдёт.

Если бы этот кастомный таймер понадобился бы в других местах игры → стоило бы его переместить в отдельный статик класс, чтобы использовать и в других местах.

Для контроля спавна волн, я создал простое деление на роли, чтобы контролировать спавн чуть более детально:

```
1 enum EnemyRole
2 {
3     Boss,
4
5     Fast_1,
6     Fast_2,
7     Fast_3,
8
9     Tank_1,
10    Tank_2,
11    Tank_3,
12
13    Swarm_1,
14    Swarm_2,
15    Swarm_3
16 }
```

Каждому мобу вручную присваивается EnemyRole, в зависимости от его характеристик. Позже это будет использоваться для генерации волн.

SpawnerManager.

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using TMPPro;
4 using UnityEngine;
5
6 public class EnemySpawnerManager : Manager<EnemySpawnerManager>
7 {
8     [Header("Demo Spawn Settings")]
9     [SerializeField] private EnemyType[] enemiesToSpawn;
10    [SerializeField] private int countPerEnemy = 2;
11    [SerializeField] private float spawnDelay = 1.33f;
12
13    [Header("Wave Timings")]
14    [SerializeField] private float preWaveDelay = 10f;
15    [SerializeField] private float postWaveDelay = 15f;
16
17    [Header("UI")]
18    [SerializeField] private GameObject timerWaveCanvas;
19    [SerializeField] private TextMeshProUGUI timerTXT;
20
21    [Header("Path Preview")]
22    [SerializeField] private GameObject skullPreviewPrefab;
23    [SerializeField] private float previewRepeatDelay = 0.7f;
24    [SerializeField] private float previewSpawnDelay = 0.02f;
25
26    private int[] aliveEnemiesPerWave;
27    private bool[] waveSpawnFinished;
28    private bool[] waveCleared;
29
30    public int CurrentWave { get; private set; } = 0;
31    private int maxWave = 5;
32
33    private SceneEnabler currentSceneData;
34    private bool isSpawning = false;
35    private Coroutine waveLoopCoroutine;
36
37    private int[] currentWavePathIndexes;
```

```
38
39 private float[] alphasPreview = { 1f, 0.65f, 0.45f };
```

SceneEnabled(int) - вызывается из GameFlowManager при загрузке сцены с уровнем. По стандарту 5 волн, но можно прокинуть любое значение. Ищем у текущей активной сцены SceneEnabler и берём его данные о путях. Вместе с тем, сразу же инициализируем WaveGenerator, чтобы тот позаботился и заранее просчитал волны врагов.

StartSpawning вызывается уже при активации состояния Playing (нажатии кнопки на пре-старт экране). Он запускает корутину WaveLoop.

```
1 // -----
2 // INITIALIZATION
3 // -----
4
5 public void SceneEnabled(int MaxWave = 5)
6 {
7     currentSceneData = FindObjectOfType<SceneEnabler>();
8
9     if (currentSceneData == null) { Debug.LogError("SceneEnabler no
10
11     maxWave = MaxWave;
12
13     HudManager.Instance.SetTotalWaveIcons(maxWave);
14
15     WaveGenerator.Instance.InitializationGenerator(maxWave, Difficu
16
17     ResetNotify(MaxWave);
18 }
19
20 // -----
21 // PUBLIC API
22 // -----
23
24 public void StartSpawning()
25 {
26     if (isSpawning) return;
27
28     if (currentSceneData == null) { Debug.LogError("No SceneEnabler
29         return;
30     }
31     isSpawning = true;
32     waveLoopCoroutine = StartCoroutine(WaveLoop());
33 }
34
35 public void StopSpawning()
36 {
37     if (!isSpawning && waveLoopCoroutine == null)
38         return;
39
40     isSpawning = false;
41     if(waveLoopCoroutine != null)
42     {
43         StopCoroutine(waveLoopCoroutine);
44         waveLoopCoroutine = null;
45     }
46
47     CurrentWave = 0;
48     HudManager.Instance.TimerWaveHide();
49 }
50
```

WaveLoop содержит while цикл, который будет крутиться, пока волны можно спавнить. Получаем пути и начинаем первую часть цикла → PreWaveDelay.

PreWaveDelay → запускает отсчёт перед спавном, а также запускает корутину PathPreviewLoop.

PathPreviewLoop → спавнит по три префаб объекта с определенным интервалом. Задержка между спавном каждого объекта - едва заметная, но достаточная, чтобы сделать эффект призрачного силуэта. К тому же, каждый следующий объект понижается в непрозрачности на ~33%. Между каждыми тремя такими объектами задержка чуть спавна чуть больше. Сделано это для того, чтобы указать маршрут, по которому скоро начнёт движение волна мобов. И пока таймер не завершился, такие объекты продолжают спавниться. Можно было сделать просто какой-нибудь эффект, сделать простую фигуру, в моём случае летают черепки.

```
1
2 // -----
3 // MAIN LOOP
4 // -----
5
6 private IEnumerator WaveLoop()
7 {
8     while (isSpawning && CurrentWave != maxWave)
9     {
10         CurrentWave++;
11         if(CurrentWave == maxWave)
12         {
13             isSpawning = false;
14         }
15
16         currentWavePathIndexes = GetPathsForWave();
17
18         HudManager.Instance.SetCurrentWaveIcon(CurrentWave);
19
20         yield return StartCoroutine(PreWaveDelay());
21
22         aliveEnemiesPerWave[CurrentWave] = 0;
23         yield return StartCoroutine(SpawnWave());
24
25         yield return StartCoroutine(PostWaveDelay());
26     }
27
28     waveLoopCoroutine = null;
29     HudManager.Instance.TimerWaveHide();
30 }
31
32 // -----
33 // PRE-WAVE
34 // -----
35
36 private IEnumerator PreWaveDelay()
37 {
38     float timer = preWaveDelay;
39     HudManager.Instance.TimerWaveShow();
40
41     Coroutine previewRoutine = StartCoroutine(PathPreviewLoop());
42 }
```

```

43     while(timer > 0f)
44     {
45         HudManager.Instance.TimerWaveSet(timer);
46
47         timer -= Time.deltaTime *
48             SpeedGameManager.Instance.SpeedMultiplier;
49
50         yield return null;
51     }
52
53     if (previewRoutine != null)
54     {
55         StopCoroutine(previewRoutine);
56     }
57
58     HudManager.Instance.TimerWaveHide();
59 }
60
61 private IEnumerator PathPreviewLoop()
62 {
63     yield return new WaitForSeconds(1f);
64     SpawnPreviewForCurrentWave();
65     yield return WaitScaled(0.3f);
66
67     while (true)
68     {
69         SpawnPreviewForCurrentWave();
70         yield return WaitScaled(previewRepeatDelay);
71     }
72 }
73
74 private void SpawnPreviewForCurrentWave()
75 {
76     if (skullPreviewPrefab == null)
77         return;
78
79     foreach (int pathIndex in currentWavePathIndexes)
80     {
81         var path = currentSceneData.enemyPaths[pathIndex].points;
82
83         if (path == null || path.Length == 0) continue;
84
85         StartCoroutine(SpawnSkullTrail(path));
86     }
87 }
88
89 private IEnumerator SpawnSkullTrail(Transform[] path)
90 {
91     for (int i = 0; i < 3; i++)
92     {
93         GameObject skull = Instantiate(skullPreviewPrefab);
94         var preview = skull.GetComponent<PathPreview>();
95         if (preview != null)
96         {
97             preview.Init(path);
98         }
99
100         var sp = skull.GetComponent<SpriteRenderer>();
101         if (sp != null)
102         {
103             Color color = sp.color;
104             color.a = alphasPreview[i];
105             sp.color = color;
106         }
107         yield return null;
108
109         yield return WaitScaled(previewSpawnDelay);

```

```

110     }
111
112 }
```

Далее следует SpawnWave цикл. Собственно, сам спавн врагов. Мы получаем из генератора по текущему индексу волны List мобов. Передаем каждого моба в метод SpawnEnemy, делаем небольшую задержку и продолжаем спавн.

PostWave - последняя часть цикла, в виде перерыва перед спавном следующей волны.

```

1 // -----
2 // SPAWN WAVE
3 // -----
4
5
6 private IEnumerator SpawnWave()
7 {
8     List<EnemyType> waveEnemies =
9         WaveGenerator.Instance.GetEnemyWaveList(CurrentWave-1);
10
11     if (waveEnemies == null || waveEnemies.Count == 0)
12     {
13         Debug.LogWarning("Wave is empty!");
14         yield break;
15     }
16
17     foreach (var enemyType in waveEnemies)
18     {
19         SpawnEnemy(enemyType, currentWavePathIndexes);
20
21         yield return WaitScaled(spawnDelay);
22     }
23     waveSpawnFinished[CurrentWave] = true;
24 }
25
26 // -----
27 // POST WAVE
28 // -----
29
30 private IEnumerator PostWaveDelay()
31 {
32     yield return WaitScaled(postWaveDelay);
33 }
34
35
```

GetPathsForWave - выдаёт маршрут для текущей волны. Если в данных SceneEnabler мы видим для текущей волны установку флага custom, значит для данной волны путь отмечен вручную. Мы просто возвращаем его. Если же путь вручную не указан, значит будем получать случайный из доступных. Если стоит галочка custom и указано два или более пути, значит для данной волны предполагается случайное распределение врагов по всем выделенным маршрутам.

GetRandomSinglePath - берёт случайный путь из доступных.

```

1 // -----
2 // PATH SELECTION
3 // -----
```



```

4
5     private int[] GetPathsForWave()
6     {
7         //Customs
8         if (currentSceneData.useCustomWaveSettings &&
9             currentSceneData.waveCustomPaths != null &&
10            CurrentWave - 1 < currentSceneData.waveCustomPaths.Length)
11         {
12             var custom = currentSceneData.waveCustomPaths[CurrentWave -
13
14         // Random
15         if (custom == null || custom.Length == 0)
16         {
17             return GetRandomSinglePath();
18         }
19
20         return custom;
21     }
22
23     return GetRandomSinglePath();
24 }
25
26 private int[] GetRandomSinglePath()
27 {
28     int index = Random.Range(0, currentSceneData.enemyPaths.Length)
29     return new int[] { index };
30 }

```

Как происходит спавн врагов? SpawnEnemy получает тип врага (вариант) и массив путей. Мы получаем из PoolManager нужный тип врага, затем получаем случайный путь из пришедших. В большинстве случаев это массив на один элемент. Если же вручную указано больше, то каждый враг может быть случайно заспавнен на одном из возможных маршрутов. В массив Transform присваиваем по индексу нужные объекты с диапазонами под точки.

Перемещаем объект в стартовую пазацию. Получаем его компоненты. Передаём врагу текущую волну (это нужно для учёта уничтоженных врагов из каждой волны). Увеличиваем массив живых врагов по индексу волны. Передаём путь в SetPath, делаем врага активным и стартуем движение.

```

1     // -----
2     // ENEMY SPAWN
3     // -----
4
5     private void SpawnEnemy(EnemyType type, int[] pathIndexes)
6     {
7         var go = PoolManager.Instance.Get(type.ToString());
8
9         if (go == null) return;
10
11         int pathIndex = pathIndexes[Random.Range(0, pathIndexes.Length)]
12
13         Transform[] path =
14             currentSceneData.enemyPaths[pathIndex].points;
15
16         if (path == null || path.Length == 0)
17         {
18             Debug.LogError($"Path {pathIndex} is empty");
19             return;

```

```

20     }
21
22
23     go.transform.position = path[0].position;
24
25     var movement = go.GetComponent<EnemyMovement>();
26     var controller = go.GetComponent<EnemyController>();
27
28     if (controller == null)
29     {
30         Debug.LogError("EnemyController is Null! Spawn canceling!")
31         return;
32     }
33     controller.SetWaveIndex(CurrentWave);
34     aliveEnemiesPerWave[CurrentWave]++;
35
36     movement.SetPath(path);
37     go.SetActive(true);
38     movement.StartMovement();
39 }
40

```

На момент написания документации я понял, что следует улучшить систему. Сейчас по двум путям можно пойти только при указании их вручную и установке флага Custom. Думаю, следует добавить флаг RandomMultiplePath[] - при true для индекса волны будет случайно выбираться - по одному пути пойдут mobs или нескольким. RandomMultiplePath[3] = true; Означает, что с 4 волны есть шанс, что mobs пойдут по нескольким путям сразу. В связи с тем, что большие карты не планируются, вряд ли будет более трёх путей на карте и ограничивать, или добавлять RandomDoublePath не имеет смысла для данного проекта.

Когда враг умирает, он вызывает метод NotifyEnemyKilled и передаём индекс волны, к которой он принадлежит. В случае, если все враги волны уничтожены, то в GameManager мы записываем +1 волну. В конце забега нам понадобится это, чтобы выводить данные об уничтоженных волнах.

К такому решению я пришёл не сразу, оно создаёт жесткую связку, но зато решение простое в реализации. Я рассматривал иные варианты, чтобы не писать сложный код, и чтобы всё работало также просто, но многие упирались в проблемы. Если хранить только мобов, то мы никогда не запишем + волну, потому что враги могут начать спавниться из второй волны до уничтожения первой. А я не хотел менять Game Flow логику. Я хочу чтобы mobs могли спавниться даже до уничтожения предыдущей волны. Можно было бы просто при начале каждой волны делать +волну. В таком случае, даже если бы волна не была уничтожена, она всё равно бы попадала в статистику. Да, погрешность от реальных

данных = 1, редко 2, волнам. Хотелось всё-таки сделать рабочую систему. Получилось такое решение.

```
1 // -----
2 // ENEMY DIE
3 // -----
4 public void NotifyEnemyKilled(int waveIndex)
5 {
6     aliveEnemiesPerWave[waveIndex]--;
7     if (waveSpawnFinished[waveIndex] &&
8         !waveCleared[waveIndex] &&
9         aliveEnemiesPerWave[waveIndex] == 0)
10    {
11        waveCleared[waveIndex] = true;
12        GameManager.Instance.WaveCleared();
13    }
14 }
15
16 private void ResetNotify(int maxWave = 5)
17 {
18     aliveEnemiesPerWave = new int[maxWave+1];
19     waveSpawnFinished = new bool[maxWave+1];
20     waveCleared = new bool[maxWave+1];
21 }
22
23 // -----
24 // CUSTOM WAIT
25 // -----
26
27 private IEnumerator WaitScaled(float duration)
28 {
29     float timer = duration;
30
31     while (timer > 0f)
32     {
33         timer -= Time.deltaTime *
34             SpeedGameManager.Instance.SpeedMultiplier;
35         yield return null;
36     }
37 }
38 }
```

WaveGenerator.

i Менеджер спавна решает только когда нужно спавнить. Кого спавнить будет решать WaveGenerator.

Лист врагов сейчас указывается вручную. В ArtefactsManager я использовал более продвинутое продакшен-стайл решение: Addressables. Позже возможно я заменю и эту часть кода, но для документации пока оставлю так.

Создаём два словаря.

Первый будет хранить роль и мобов, что относятся к данной роли. Например: tank_1: Plant1, Goblin1, Rot.

Второй будет хранить тип каждого моба и его EnemyStats (для вычислений).

InitMap - инициализируем карты. С каждого префаба получаем его EnemyStats, EnemyType, EnemyRoleTier и связываем всё это дело в две удобные карты.

InitializationGenerator(int) - инициализируем генератор, передавая в него максимальное число волн, а также вес волн для текущего этажа (берётся из DifficultyManager). Для каждой волны вызываем BuildWave(). В итоге у нас должен будет получиться массив из нескольких листов врагов.

```
1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class WaveGenerator : Manager<WaveGenerator>
5 {
6     [Header("EnemyPrefabs")]
7     [SerializeField] private List<GameObject> enemyPrefabs;
8
9     private Dictionary<EnemyRoleTier, List<EnemyType>> roleMap;
10    private Dictionary<EnemyType, EnemyStats> statsMap;
11
12    private List<List<EnemyType>> waves;
13    private List<int> waveBudgets;
14
15    private int maxWaveCount = 5;
16
17    private void Start()
18    {
19        InitMap();
20    }
21
22    private void InitMap()
23    {
24        statsMap = new();
25        roleMap = new();
26
27        foreach (var prefab in enemyPrefabs)
28        {
29            var stats = prefab.GetComponent<EnemyStats>();
30
31            statsMap[stats.type] = stats;
32
33            if (!roleMap.ContainsKey(stats.role))
34            {
35                roleMap[stats.role] = new List<EnemyType>();
36            }
37            roleMap[stats.role].Add(stats.type);
38        }
39    }
40
41    public void InitializationGenerator(int maxWave, int weightFloor)
42    {
43        maxWaveCount = maxWave;
44
45        waves = new List<List<EnemyType>>(maxWave);
46
47        waveBudgets = BuildWaveBudgets(maxWave, weightFloor);
48
49        for (int wave = 0; wave < maxWave; wave++)
50        {
51            waves.Add(BuildWave(wave));
52        }
53    }
```

BuildWaveBudgets - возвращает лист веса под каждую волну. Пока что используется вычисление ниже: мы берём общий вес, умножаем на текущую волну и делим на sum (треугольное число, для 5 = 1+2+3+4+5 = 15). В среднем градация веса волны получается приемлемой, но в будущем возможны правки.

BuildWave - получает индекс волны. Веса мы уже записали в массив и будем работать с ними по индексам волн. Генерируем модификаторы через GenerateModifiers. Далее делаем проверку на IsBossWave, а также проверяем есть ли вообще в карте на роли босса какие-нибудь префабы. Если это волна последняя, то она в любом случае получит босса.

Начинаем сборку: будем собирать волну, пока не кончиться бюджет. Получаем роль, если роль открыта - получаем врага по роли. Если роль закрыта для текущей волны → пропускаем итерацию. Получаем вес моба из EnemyStats (Панее это было Exp врага (здоровье * скорость + урон), но в итоге я перешёл на абстрактную модель веса, которая вписывается руками и работает не от формул).

Теперь прокидываем роль внутрь GetGroupCount, чтобы получить не просто одного моба, а пачку (так волна смотрится красивее и читаемее). Вычитаем из веса волны вес каждого и добавляем их в лист.

```
1 private List<int> BuildWaveBudgets(int maxWave, int totalWeight)
2 {
3     List<int> result = new();
4
5     int sum = 0;
6     for(int i = 1; i <= maxWave; i++)
7     {
8         sum += i;
9     }
10
11     int allocated = 0;
12
13     for (int i = 1; i <= maxWave; i++)
14     {
15         int budget = totalWeight * i / sum; //3000: 200, 400, 600,
16         result.Add(budget);
17         allocated += budget;
18     }
19
20     result[result.Count - 1] += totalWeight - allocated;
21     return result;
22 }
23
24 private List<EnemyType> BuildWave(int waveIndex)
25 {
26     int budget = waveBudgets[waveIndex];
27     List<EnemyType> result = new();
28
29     HashSet<WaveModifier> modifiers =
30         GenerateModifiers(waveIndex);
31
32     if (IsBossWave(waveIndex) && roleMap.ContainsKey(EnemyRoleTier.
33     {
34         EnemyType boss = PickBoss();
35         int bossCost = statsMap[boss].cost;
36     }
```

```

37         if (bossCost <= budget)
38         {
39             result.Add(boss);
40             budget -= bossCost;
41         }
42     }
43
44     int safetyCounter = 3000;
45
46     while (budget > 0 &&
47         safetyCounter-- > 0 &&
48         HasEnemyFitBudget(budget))
49     {
50         EnemyRoleTier role = PickRoleByWave(waveIndex);
51
52         // роль открыта.
53         if (!IsRoleUnlocked(role, waveIndex))
54             continue;
55
56         // для роли есть враги.
57         if (!roleMap.ContainsKey(role) || roleMap[role].Count == 0)
58             continue;
59
60         EnemyType type = PickEnemy(role);
61         int cost = statsMap[type].cost;
62
63         // Если не помещается в бюджет
64         if (cost > budget)
65             continue;
66
67
68
69         int count = GetGroupCount(
70             role,
71             waveIndex,
72             modifiers);
73
74         for (int i=0; i<count; i++)
75         {
76             if (cost > budget)
77             {
78                 break;
79             }
80             result.Add(type);
81             budget -= cost;
82         }
83     }
84
85     return result;
86 }

```

PickRoleByWave - сортирует роли по этажам. К примеру, на первом этаже есть только враги с ролью Swarm_1 (рой). После вычисления роли берём из карты roleMap случайного юнита подходящей роли.

PickEnemy - возвращаем EnemyType нужного моба.

IsRoleUnlocked(EnemyRoleTier, int) - проверяет доступна ли роль для текущей волны. Это ограничение ролей, чтобы в первых волнах не могли спавниться самые живучие и быстрые враги.

HasEnemyFitBudget - проверяет, существует ли хотя бы ещё один моб, которого можно купить на оставшийся бюджет. Если да - возвращает true и мы продолжаем цикл заполнения волны.

```
1  //////////////////////////////////////
2  /// Role Selection
3  //////////////////////////////////////
4
5  private EnemyRoleTier PickRoleByWave(int wave)
6  {
7      int roll = Random.Range(0, 100);
8
9      if (wave == 0)
10         return EnemyRoleTier.Swarm_1;
11
12     if (wave <= 1)
13     {
14         if (roll < 70) return EnemyRoleTier.Swarm_1;
15         if (roll < 90) return EnemyRoleTier.Tank_1;
16         return EnemyRoleTier.Fast_1;
17     }
18
19     if (wave <= 2)
20     {
21         if (roll < 50) return EnemyRoleTier.Swarm_2;
22         if (roll < 75) return EnemyRoleTier.Tank_2;
23         return EnemyRoleTier.Fast_2;
24     }
25
26     return (EnemyRoleTier)Random.Range(0, roleMap.Count);
27 }
28
29 //////////////////////////////////////
30 /// Helpers
31 //////////////////////////////////////
32 private bool IsBossWave(int wave)
33 {
34     return wave == maxWaveCount - 1;
35 }
36
37 private EnemyType PickEnemy(EnemyRoleTier role)
38 {
39     var pool = roleMap[role];
40     return pool[Random.Range(0, pool.Count)];
41 }
42
43 private EnemyType PickBoss()
44 {
45     var bosses = roleMap[EnemyRoleTier.Boss];
46     return bosses[Random.Range(0, bosses.Count)];
47 }
48
49 private bool IsRoleUnlocked(EnemyRoleTier role, int wave)
50 {
51     return role switch
52     {
53         EnemyRoleTier.Swarm_1 => true,
54         EnemyRoleTier.Tank_1 => wave >= 0,
55         EnemyRoleTier.Fast_1 => wave >= 0,
56
57         EnemyRoleTier.Tank_2 => wave >= 1,
58         EnemyRoleTier.Swarm_2 => wave >= 1,
59
60         EnemyRoleTier.Tank_3 => wave >= 2,
```

```

61         EnemyRoleTier.Fast_2 => wave >= 2,
62         EnemyRoleTier.Swarm_3 => wave >= 2,
63
64         EnemyRoleTier.Fast_3 => wave >= 3,
65
66         EnemyRoleTier.Boss => wave >= 4,
67
68         _ => false
69     };
70 }
71
72 private bool HasEnemyFitBudget(int budget)
73 {
74     foreach (var stats in statsMap.Values)
75     {
76         if (stats.cost <= budget)
77         {
78             return true;
79         }
80     }
81     return false;
82 }

```

GetGroupCount(EnemyRoleTier, int, HashSet<WaveModifier>) - главный сборщик волны. Сначала он обрабатывает модификаторы, которые могут выпасть для волны, а затем проверяет на выпавшую роль и выдаёт случайное количество мобов в зависимости от волны. Например: на 5 волне выпавшая роль танка второго уровня подразумевает спавн толпы из 3-6 мобов. Таким образом собираются небольшие толпы, все они суммируются и получается красивая волна.

Этот метод придётся расширять, по мере появления нового функционала и новых способов группировочной сборки. Условно, если захочется добавить хаотичные модификаторы, или логику под большее количество волн - код разрастётся.

```

1  //////////////////////////////////////
2  /// Grouping
3  //////////////////////////////////////
4  private int GetGroupCount(EnemyRoleTier role, int wave, HashSet<Wa
5  {
6      if (modifiers.Contains(WaveModifier.SwarmRush))
7      {
8          if (role == EnemyRoleTier.Swarm_1)
9          {
10             return Random.Range(8, 14);
11          }
12          if (role == EnemyRoleTier.Swarm_2)
13          {
14             return Random.Range(7, 12);
15          }
16          if (role == EnemyRoleTier.Swarm_3)
17          {
18             return Random.Range(5, 10);
19          }
20      }
21
22      if (modifiers.Contains(WaveModifier.TankOverflow))
23      {
24          if (role == EnemyRoleTier.Tank_1)
25             return Random.Range(6, 12);

```



```

26
27         if (role == EnemyRoleTier.Tank_2)
28             return Random.Range(5, 10);
29     }
30
31     if (modifiers.Contains(WaveModifier.FastRush))
32     {
33         if (role == EnemyRoleTier.Fast_1)
34             return Random.Range(6, 8);
35
36         if (role == EnemyRoleTier.Fast_2)
37             return Random.Range(4, 7);
38     }
39
40     switch (role)
41     {
42         case EnemyRoleTier.Swarm_1:
43             return Random.Range(4, 8);
44         case EnemyRoleTier.Swarm_2:
45             return Random.Range(3, 7);
46         case EnemyRoleTier.Swarm_3:
47             return Random.Range(3, 6);
48
49
50         case EnemyRoleTier.Tank_1:
51             if (wave <= 1)
52             {
53                 return Random.Range(1, 3);
54             }
55             if (wave <= 3)
56             {
57                 return Random.Range(3, 6);
58             }
59             return Random.Range(2, 4);
60
61         case EnemyRoleTier.Tank_2:
62             if (wave <= 2)
63             {
64                 return Random.Range(2, 3);
65             }
66             if (wave <= 4)
67             {
68                 return Random.Range(3, 6);
69             }
70             return Random.Range(2, 3);
71
72         case EnemyRoleTier.Tank_3:
73             return 1;
74
75         case EnemyRoleTier.Fast_1:
76             if (wave <= 2)
77             {
78                 return Random.Range(2, 4);
79             }
80             if (wave <= 4)
81             {
82                 return Random.Range(4, 8);
83             }
84             return Random.Range(2, 4);
85
86         case EnemyRoleTier.Fast_2:
87             if (wave <= 2)
88             {
89                 return Random.Range(1, 3);
90             }
91             if (wave <= 4)
92             {

```

```

93         return Random.Range(3, 5);
94     }
95     return Random.Range(1, 2);
96
97     case EnemyRoleTier.Fast_3:
98         return 1;
99
100 }
101 return 1;
102 }

```

К началу игры генератор уже сгенерировал массив листов с порядком спавна. Если ничего не случилось, никаких ошибок нет, то мы вернём по запросу List мобов по индексу волны.

Ниже следует GenerateModifiers(int) - который случайно может выкинуть модификатор, и в таком случае фолна получит определенный паттерн. Например: волна танков.

```

1  //////////////////////////////////////
2  /// GETING
3  //////////////////////////////////////
4  public List<EnemyType> GetEnemyWaveList(int currentWave)
5  {
6      if (waves == null || currentWave < 0 || currentWave >= waves.Co
7      {
8          Debug.LogError("WavesList is Null!");
9          return new List<EnemyType>();
10     }
11     return waves[currentWave];
12 }
13
14
15 private HashSet<WaveModifier> GenerateModifiers(int wave)
16 {
17     HashSet<WaveModifier> modifiers = new();
18
19     if (Random.value < 0.08f)
20     {
21         modifiers.Add(WaveModifier.TankOverflow);
22     }
23
24     if (Random.value < 0.16f)
25     {
26         modifiers.Add(WaveModifier.SwarmRush);
27     }
28
29     if (Random.value < 0.06f)
30     {
31         modifiers.Add(WaveModifier.FastRush);
32     }
33
34     return modifiers;
35 }
36
37 }
38
39
40 public enum WaveModifier
41 {
42     TankOverflow,
43     SwarmRush,
44     FastRush
45 }

```

-  Функционал спавнера можно долго расширять. Бесконечность предполагает составление случайных волн, однако необходимо и наличие паттернов. Как например - спавн танков в первой волне в размере 1-3. Такие паттерны можно добавлять долго и увеличивать разнообразие генерации.
-  На данном этапе волна получается приемлемой и играбельной. Выглядит красиво, по виду чувствуется сбалансированной. В процессе финального баланса что-то может измениться, но результатом я доволен.